

T-Sharp (T#) — Temel Kavramlar

İçindekiler

1. Merhaba Dünya & Yazdır
2. Değişkenler ve Veri Tipleri
3. Değişken Güncelleme ve Operatörler
4. Kullanıcıdan Girdi Alma
5. Listeler
6. Sözlükler
7. Koşullar — `eger` / `degilse`
8. Döngüler — `dongu` ve `her`
9. Fonksiyonlar
10. Yerleşik Fonksiyonlar (Hazır Araçlar)
11. Yorumlar

1. Merhaba Dünya & Yazdır

T#'da ekrana çıktı vermek için `yazdır` (ya da kısaca `yaz`) komutu kullanılır.

```
yazdır "Merhaba, Dünya!"
```

```
yaz "T-Sharp ile programlama çok kolay!"
```

Birden fazla şeyi yan yana yazdırmak için aralarına boşluk bırakarak yazabilirsiniz:

```
degisken isim = "Talha"  
yazdır "Merhaba," isim
```

Çıktı:

```
Merhaba, Talha
```

Boş satır yazdırmak için sadece `yazdır` yazmanız yeterlidir:

```
yazdır "Birinci satır"  
  
yazdır  
  
yazdır "Üçüncü satır"
```

2. Değişkenler ve Veri Tipleri

T#'da değişken tanımlamak için `degisken` anahtar kelimesi kullanılır. Tür belirtmenize **gerek yoktur** — sistem otomatik algılar.

Söz dizimi

```
degisken <ad> = <deger>
```

Veri Tipleri

Tip	T# Karşılığı	Örnek Değer
Metin (string)	<code>"..."</code> veya <code>'...'</code>	<code>"Merhaba"</code>
Tam Sayı	doğrudan sayı	<code>42</code>
Ondalıklı	noktalı sayı	<code>3.14</code>
Mantıksal	<code>dogru</code> / <code>yanlis</code>	<code>dogru</code>
Boş	<code>hic</code>	<code>hic</code>

Örnekler

```
// Metin
degisken isim = "Ahmet"

// Tam sayı
degisken yas = 18

// Ondalıklı sayı
degisken pi = 3.14159

// Mantıksal (boolean)
degisken aktif = dogru
degisken pasif = yanlis

// Boş değer
degisken sonuc = hic

// Hepsini yazdır
yazdır isim
yazdır yas
yazdır pi
yazdır aktif
yazdır sonuc
```

Çıktı:

```
Ahmet
18
3.14159
dogru
hic
```

Not: Metin değerleri `"` veya `'` içine alınmalıdır. Sayılar ve `dogru` / `yanlis` / `hic` tırnak gerektirmez.

3. Değişken Güncelleme ve Operatörler

Tanımlanmış bir değişkeni güncellemek için `degisken` yazmaya gerek yoktur, doğrudan atama yapabilirsiniz.

```
degisken sayi = 10
sayi = 20
yazdir sayi
```

Aritmetik Operatörler

Operatör	Anlamı	Örnek
+	Toplama	5 + 3 → 8
-	Çıkarma	10 - 4 → 6
*	Çarpma	6 * 7 → 42
/	Bölme	15 / 4 → 3.75
// veya bolum	Tam bölme	15 // 4 → 3
% veya mod	Kalan	15 % 4 → 3
**	Üs alma	2 ** 8 → 256

Bileşik Atama Operatörleri

```
degisken x = 10

x += 5      // x artık 15
x -= 3      // x artık 12
x *= 2      // x artık 24
x /= 4      // x artık 6.0
x **= 2     // x artık 36.0
```

Karşılaştırma Operatörleri

Türkçe	Sembol	Anlamı
esittir	==	Eşit mi?
degildir	!=	Eşit değil mi?
buyuktur	>	Büyük mü?
kucuktur	<	Küçük mü?
buyuk_esit	>=	Büyük eşit mi?
kucuk_esit	<=	Küçük eşit mi?

Mantıksal Operatörler

Türkçe	Anlamı
ve	İkisi de doğruysa
veya	En az biri doğruysa
degil	Tersini al

```
degisken a = 10
degisken b = 20

yazdır a buyuktur b      // yanlis
yazdır a kucuktur b      // dogru
yazdır a esittir 10       // dogru
yazdır a esittir 10 ve b esittir 20  // dogru
```

4. Kullanıcıdan Girdi Alma

Kullanıcının klavyeden veri girmesini sağlamak için `girdi` komutu kullanılır.

Söz dizimi

```
girdi <degisken_adi> "Ekranda görünecek mesaj"
```

Örnekler

```
girdi isim "Adınızı girin: "
yazdır "Merhaba," isim
```

```
girdi yas_metin "Yaşınızı girin: "
degisken yas = sayiya(yas_metin)
yazdır "Doğum yılınız:" 2025 - yas
```

Not: `girdi` ile alınan değerler her zaman **metin** (string) olarak gelir. Sayı işlemi yapmak için `sayiya()` fonksiyonuyla dönüştürmelisiniz.

5. Listeler

Listeler, birden fazla değeri sıralı biçimde tutan yapılardır.

Tanımlama

```
liste <ad> [eleman1, eleman2, eleman3]
```

Örnekler

```
liste meyveler ["elma", "muz", "kiraz"]  
liste sayilar [1, 2, 3, 4, 5]  
liste karisik [42, "merhaba", dogru, 3.14]  
liste bos []
```

Elemanlara Erişim (İndeks)

İndeksler **o'dan** başlar.

```
liste renkler ["kırmızı", "yeşil", "mavi"]  
  
yazdır renkler[0]    // kırmızı  
yazdır renkler[1]    // yeşil  
yazdır renkler[2]    // mavi
```

Eleman Güncelleme

```
liste sayilar [10, 20, 30]  
sayilar[1] = 99  
yazdır sayilar[1]    // 99
```

Liste Fonksiyonları

```
liste meyveler ["elma", "muz"]

// Eleman ekle
ekle(meyveler, "kivi")
yazdır uzunluk(meyveler)    // 3

// Eleman çıkar
cikar(meyveler, "muz")

// Sırala
liste sonuc = sirala(meyveler)

// Tersine çevir
liste ters = tersine_cevir(meyveler)

// İçinde mi?
yazdır icindemi("elma", meyveler)    // dogru

// Uzunluk
yazdır uzunluk(meyveler)

// Toplam (sayı listesi)
liste notlar [70, 85, 90, 95]
yazdır toplam(notlar)    // 340
yazdır ortalama(notlar)    // 85.0
yazdır minimum(notlar)    // 70
yazdır maksimum(notlar)    // 95
```

Dilimleme

```
liste harfler ["a", "b", "c", "d", "e"]
liste dilim = dilimleme(harfler, 1, 4)
// dilim = ["b", "c", "d"]
```

6. Sözlükler

Sözlükler, **anahtar: değer** çiftleriyle veri tutan yapılardır (Python'daki **dict** gibi).

Tanımlama

```
sozluk <ad> {anahtar1: deger1, anahtar2: deger2}
```

Örnekler

```
sozluk ogrenci {"isim": "Talha", "yas": 20, "not": 95}

yazdir ogrenci["isim"]    // Talha
yazdir ogrenci["yas"]     // 20
```

Değer Güncelleme

```
sozluk urun {"ad": "Kalem", "fiyat": 5}
urun["fiyat"] = 8
yazdir urun["fiyat"]     // 8
```

Yeni Anahtar Ekleme

```
sozluk kisi {"ad": "Ali"}
kisi["sehir"] = "Ankara"
yazdir kisi["sehir"]     // Ankara
```

7. Koşullar — **eger** / **degilse**

Söz dizimi

```
eger <koşul>:
    // koşul doğruysa çalışır
degilse:
    // koşul yanlışsa çalışır
son
```

Önemli: Her **eger** bloğu **son** ile kapatılmalıdır!

Basit Örnek

```
degisken yas = 20

eger yas buyuk_esit 18:
    yazdır "Reşitsiniz, giriş yapabilirsiniz."
degilse:
    yazdır "Üzgünüz, henüz reşit değilsiniz."
son
```

degilse Olmadan

```
degisken not = 45

eger not kucuktur 50:
    yazdır "Sınıfta kaldınız."
son
```

İç İç Koşullar

```
degisken puan = 75

eger puan buyuk_esit 90:
    yazdır "Pekiyi"
degilse:
    eger puan buyuk_esit 70:
        yazdır "İyi"
    degilse:
        eger puan buyuk_esit 50:
            yazdır "Orta"
        degilse:
            yazdır "Zayıf"
        son
    son
son
```

Mantıksal Operatörlerle Koşul

```
degisken a = 5
degisken b = 10

eger a buyuktur 0 ve b kucuktur 20:
    yazdır "İkisi de geçerli!"
son

eger a esittir 5 veya b esittir 5:
    yazdır "En az biri 5'e eşit."
son
```

8. Döngüler — **dongu** ve **her**

8.1 Koşullu Döngü — **dongu**

dongu komutu, belirtilen koşul doğru olduğu sürece çalışmaya devam eder.

```
dongu <koşul>:
    // tekrarlanan kodlar
son
```

Örnek — 1'den 5'e say:

```
degisken sayac = 1

dongu sayac kucuk_esit 5:
    yazdır sayac
    sayac += 1
son
```

Çıktı:

```
1
2
3
4
5
```

8.2 For Each Döngüsü — `her`

`her` komutu bir listedeki her elemana sırayla erişir.

```
her <degisken> icinde <liste>:
    // her eleman için çalışır
son
```

Örnek — Liste elemanları:

```
liste sehirler ["Ankara", "İstanbul", "İzmir"]

her sehir icinde sehirler:
    yazdır sehir
son
```

Çıktı:

```
Ankara
İstanbul
İzmir
```

Örnek — Sayı aralığı:

```
// 0'dan 4'e kadar (0, 1, 2, 3, 4)
her i icinde [0, 1, 2, 3, 4]:
    yazdır i
son
```

8.3 Döngü Kontrolü — `dur` ve `devam`

Komut	Anlamı
<code>dur</code>	Döngüyü tamamen durdur (break)
<code>devam</code>	Bu iterasyonu atla, devam et (continue)

```
// Sadece çift sayıları yazdır, 8'e gelince dur
degisken i = 0

dongu i kucuktur 20:
    i += 1
    eger i mod 2 esittir 1:
        devam
    son
    eger i esittir 8:
        dur
    son
    yazdır i
son
```

Çıktı:

```
2
4
6
```

9. Fonksiyonlar

Fonksiyonlar, tekrar kullanılabilir kod bloklarıdır.

Tanımlama

```
fonksiyon <ad>(<parametreler>):
    // fonksiyon gövdesi
    dondur <deger>
son
```

Basit Fonksiyon

```
fonksiyon selam():  
    yazdır "Merhaba! T# öğreniyorum."  
son  
  
// Çağrı:  
selam()
```

Parametrelili Fonksiyon

```
fonksiyon topla(a, b):  
    degisken sonuc = a + b  
    dondur sonuc  
son  
  
degisken cevap = topla(15, 27)  
yazdır cevap // 42
```

Çok Parametrelili Fonksiyon

```
fonksiyon dikdortgen_alan(en, boy):  
    dondur en * boy  
son  
  
yazdır dikdortgen_alan(5, 8) // 40
```

Değer Döndürmeyen Fonksiyon

```
fonksiyon bilgi_yaz(isim, yas):  
    yazdır "İsim:" isim  
    yazdır "Yaş:" yas  
son  
  
bilgi_yaz("Talha", 20)
```

Özyinelemeli Fonksiyon (Recursive)

```
fonksiyon faktoriyel(n):  
    eger n kucuk_esit 1:  
        dondur 1  
    son  
    dondur n * faktoriyel(n - 1)  
son  
  
yazdır faktoriyel(5)    // 120
```

10. Yerleşik Fonksiyonlar (Hazır Araçlar)

T# birçok hazır fonksiyonla gelir. İşte en sık kullanılanları:

Tür Dönüştürme

Fonksiyon	Açıklama	Örnek
yaziya(x)	Herhangi bir değeri metne çevirir	yaziya(42) → "42"
sayiya(x)	Metni tam sayıya çevirir	sayiya("10") → 10
ondalikliya(x)	Metni ondalıklıya çevirir	ondalikliya("3.5") → 3.5
tur(x)	Değerin tipini söyler	tur(42) → "int"

Matematik

Fonksiyon	Açıklama	Örnek
mutlak(x)	Mutlak değer	mutlak(-5) → 5
karekok(x)	Karekök	karekok(16) → 4.0
yuvarla(x, n)	Yuvarla	yuvarla(3.567, 2) → 3.57
taban(x)	Aşağı yuvarla	taban(3.9) → 3
tavan(x)	Yukarı yuvarla	tavan(3.1) → 4
us(a, b)	Üs alma	us(2, 10) → 1024.0
rastgele_sayi(a, b)	a-b arası rastgele tam sayı	rastgele_sayi(1, 6)

Metin İşlemleri

Fonksiyon	Açıklama	Örnek
<code>uzunluk(x)</code>	Uzunluk	<code>uzunluk("merhaba")</code> → 7
<code>buyuk_harf(x)</code>	Büyük harfe çevir	<code>buyuk_harf("ali")</code> → "ALİ"
<code>kucuk_harf(x)</code>	Küçük harfe çevir	<code>kucuk_harf("ALİ")</code> → "ali"
<code>basa_bas(x)</code>	Baştaki/sondaki boşlukları sil	<code>basa_bas(" ali ")</code> → "ali"
<code>bol(metin, ayrac)</code>	Metni parçalara böl	<code>bol("a,b,c", ",")</code> → ["a","b","c"]
<code>birlestir(ayrac, liste)</code>	Listeyi birleştir	<code>birlestir("-", ["a","b"])</code> → "a-b"
<code>degistir(metin, eski, yeni)</code>	Değiştir	<code>degistir("merhaba", "a", "e")</code>
<code>bul(metin, aranan)</code>	Konum bul	<code>bul("merhaba", "ha")</code> → 3
<code>dilim(metin, bas, son)</code>	Alt metin al	<code>dilim("merhaba", 0, 3)</code> → "mer"
<code>baslar_mi(metin, on)</code>	Başlıyor mu?	<code>baslar_mi("merhaba", "mer")</code> → doğru
<code>biter_mi(metin, son)</code>	Bitiyor mu?	<code>biter_mi("merhaba", "ba")</code> → doğru

11. Yorumlar

T#'da yorum satırları `//` ile başlar. Derleyici bu satırları görmezden gelir.

```
// Bu bir yorum satırıdır, çalışmaz

degisken x = 42    // Satır sonu yorumu da desteklenir

// Aşağıdaki kod bir döngü örneğidir
dongu x buyuktur 0:
    yazdır x
    x -= 10
son
```

Tam Örnek Program

Aşağıdaki program yukarıdaki kavramların çoğunu bir arada kullanmaktadır:

```
// =====  
// T-Sharp v4.1 - Temel Kavramlar Demo Programı  
// =====  
  
// 1. Değişkenler  
degisken program_adi = "Not Hesaplayıcı"  
degisken versiyon = 1  
  
yazdır "=====  
yazdır program_adi  
yazdır "=====  
yazdır  
  
// 2. Kullanıcıdan veri al  
girdi ogrenci_adi "Öğrenci adı: "  
girdi not1_metin "1. not: "  
girdi not2_metin "2. not: "  
girdi not3_metin "3. not: "  
  
// 3. Tür dönüştür  
degisken not1 = ondalikliya(not1_metin)  
degisken not2 = ondalikliya(not2_metin)  
degisken not3 = ondalikliya(not3_metin)  
  
// 4. Liste oluştur  
liste notlar [not1, not2, not3]  
  
// 5. Ortalama hesaplama fonksiyonu  
fonksiyon ortalama_hesapla(liste_notlar):  
    degisken toplam = 0  
    her n icinde liste_notlar:  
        toplam += n  
    son  
    dondur toplam / uzunluk(liste_notlar)  
son  
  
// 6. Hesapla ve göster  
degisken ort = ortalama_hesapla(notlar)  
degisken yuvarlanmis = yuvarla(ort, 2)  
  
yazdır  
yazdır "--- SONUÇ ---"
```



```
yazdır "Öğrenci:" ogrenci_adi
yazdır "Notlar:" not1 not2 not3
yazdır "Ortalama:" yuvarlanmis

// 7. Koşullu harf notu
eger ort buyuk_esit 90:
    yazdır "Harf Notu: AA"
degilse:
    eger ort buyuk_esit 75:
        yazdır "Harf Notu: BA"
    degilse:
        eger ort buyuk_esit 60:
            yazdır "Harf Notu: BB"
        degilse:
            eger ort buyuk_esit 50:
                yazdır "Harf Notu: CC"
            degilse:
                yazdır "Harf Notu: FF (Kaldı)"
            son
        son
    son
son
```

Sonraki bölüm: `02_DOSYA_ISLEMLERI.md` — Dosya okuma, yazma ve ZIP işlemleri.